App web Yassine

DOCUMENT DE MÉTHODOLOGIE

Application web de calibration « à la main »

pricers COS et PROJ en JavaScript, douze compositions de modèles, 270 jours et 529 jeux calibrés embarqués

Ce document décrit l'outil de calibration qui fonctionne sur un poste isolé, sans Python ni serveur : architecture de la page, données embarquées, moteur de valorisation, composition des modèles, perte, optimiseur, enregistrement des réglages, reconstruction et validation.

Référence	APP-WEB-YASSINE
Version	1.0 – version finale
Date	octobre 2026
Projet	App web Yassine – calibration d'options Bitcoin (Deribit)
Auteur	Mohammed-Yassine Barnicha

Contrôle du document

Version	Date	Auteur	Objet
1.0	10/2026	M.-Y. Barnicha	Création du document : architecture, moteur, compositions, optimiseur, données, validation.

Conventions. Les nombres sont écrits à la française (virgule décimale). Sauf mention contraire, les temps sont en années (ACT/365), les prix en dollars, les volatilités en valeur décimale et les écarts de volatilité en points de base (1 bp $= 10^{-4}$ en volatilité). Les chemins de fichiers sont relatifs à la racine du projet.

Table des matières

1 Objet et périmètre **3**

2 Architecture **3**

3 Moteur de valorisation **3**

3.1 Composition des modèles 3

3.2 Pricers 4

4 Perte, optimiseur et empilement **4**

5 Jeux de paramètres, enregistrement et export **4**

6 Données et reconstruction **5**

7 Validation **5**

8 Domaine de validité **5**

1 Objet et périmètre

Besoin. Utiliser la chaîne de calibration sur des postes sans environnement Python, et rendre visibles les choix qu'elle effectue (en particulier la direction mal déterminée).

Réponse. Une page HTML autonome : les pricers COS et PROJ sont réécrits en JavaScript, les instantanés et les futures sont embarqués. Aucune installation ; un navigateur Chrome ou Edge suffit (seuls à écrire dans un dossier local). L'application ne fait pas de recherche globale lourde par défaut : elle part d'un point de production ou d'un réglage et le polit localement.

Une seule page, `calibration_a_la_main`, couvre tous les modèles de `pricers_fourier` : trois blocs de variance × quatre lois de sauts (dont Bates et Bates bi-log-normal), 270 jours (14/01/2024 – 12/10/2026).

Synthèse

Validation du moteur : $C(K{=}F) - P(K{=}F) = 0{,}0000$ bp sur les 28 compositions (martingalité) ; Black-Scholes retrouvé à moins de 0,005 bps ; écart COS–PROJ nul sur un smile ; pricer JavaScript égal à `pricers_fourier` à $6 \cdot 10^{-6}$ \$ près.

2 Architecture

- `index.html` : la page (HTML, CSS, logique) assemblée par `sources/build.py` à partir de `sources/template_full.html` et des modules JavaScript (`pricer.js`, …).
- `jours/AAAA-MM-JJ.js` : un jour = la chaîne complète déjà préparée (strikes, échéances, F, r, prix et volatilités bid/ask/mark, vega) et les points de départ connus (environ 120 Ko). La page les charge à la demande : garder le dossier `jours/` à côté de `index.html`.
- `futures.js` : historique de 286 échéances (118 futures cotés, 168 forwards implicites relevés dans les instantanés).
- `sources/telecharger_futures.py` : met à jour `futures.js`.
- `sources/ajouter_jours.py` : ajoute des jours depuis des instantanés Deribit.

La page n'exécute que du JavaScript. Le script d'ajout de jours utilise deux paquets inclus dans `sources/` : `donnees_deribit` (préparation des instantanés) et `pricers_fourier` (inversion de Black-76).

3 Moteur de valorisation

3.1 Composition des modèles

Proposition 3.1. *Si le processus de sauts (loi de taille ν, intensité λ) est indépendant de (W, v), alors* $\log \phi_{\mathrm{r}}(u) = A_{\mathrm{var}}(u,T) + T(\psi_J(u) - \mathrm{i}u\,\omega)$, $\psi_J(u) = \lambda(\widehat{\nu}(u) - 1)$, $\omega = \psi_J(-\mathrm{i}) = \lambda\kappa_J$.

Démonstration. $\log(F_T/F_0) = X_T^c + X_T^J - \omega T$; par indépendance la fonction caractéristique est le produit ; Lévy–Khintchine donne $\mathbb{E}\mathrm{e}^{\mathrm{i}uX_T^J} = \exp(\lambda T(\widehat{\nu}(u) - 1))$; $\mathbb{E}\mathrm{e}^{X_T} = 1$ impose $\omega = \lambda\kappa_J$. Deux facteurs CIR indépendants donnent la somme de deux exposants de Heston. □

Bloc	Choix	Exposant ajouté à $\log \phi_{\mathrm{r}}(u)$	c_4 pour PROJ
Variance	Black–Scholes	$-\frac{1}{2}\varsigma^2 T(u^2+\mathrm{i}u)$	0
	Heston	$C(u,T)+D(u,T)v_0$	0
	Double Heston	somme de deux exposants de Heston	0
Sauts	aucun	0	0
	Merton	$\lambda T(\mathrm{e}^{\mathrm{i}u\mu_J-\frac{1}{2}\sigma_J^2u^2}-1-\mathrm{i}u\kappa_J)$	$\lambda T(\mu_J^4+6\mu_J^2\sigma_J^2+3\sigma_J^4)$
	Bi-log-normal	somme de deux termes de Merton	somme
	Kou	$\lambda T(\frac{p\eta_1}{\eta_1-\mathrm{i}u}+\frac{(1-p)\eta_2}{\eta_2+\mathrm{i}u}-1-\mathrm{i}u\kappa_J)$, $\eta_1>1$	$24\lambda T(\frac{p}{\eta_1^4}+\frac{1-p}{\eta_2^4})$

Les deux méthodes n'ont besoin que de ϕ_{r} (fermée pour toutes les compositions), de c_1, c_2 par différences finies et, pour PROJ, de c_4 (fermé par famille de sauts). La convergence est assurée par le bloc de variance ; le bloc de sauts ne peut pas l'empêcher ($|\mathrm{e}^{T\psi_J}| \le 1$). Le moteur accepte aussi CGMY, Variance Gamma et NIG (contraintes d'existence du compensateur portées par les bornes).

3.2 Pricers

COS ($N = 512$, $L = 24$) et PROJ ($N = 2^{11}$, $L = 20$ par défaut), réglables, mêmes algorithmes que la version Python ; inversion de Black-76 vectorisée. PROJ est le pricer par défaut : fiable quand $|\mu_J|$, σ_J ou λ sont grands.

4 Perte, optimiseur et empilement

— **Perte** : cinq mesures (prix absolu, prix relatif, IV, …) × cinq pondérations, dont celle de production (`inv_vega2_equal`) ; la formule minimisée est affichée.

— **Nelder–Mead** dans l'espace des curseurs $t \in [0,1]^n$ (chaque paramètre libre ramené à sa plage) : simplexe initial de pas 0,04, réflexion, expansion, contraction, rétrécissement ; tolérance 10^{-6}, 2500 évaluations au plus. Paramètres verrouillés exclus ; sortie de $[0,1]$ ou violation de Feller $\Rightarrow f = +\infty$.

— **Jours empilés** : N jours consécutifs calibrés ensemble ; un même contrat vu deux jours de suite donne deux échéances (T puis $T - 1/365$). θ, ρ, v_0, λ_1, λ_2 peuvent être libérés jour par jour.

— **Usage diagnostique** : verrouiller $\kappa, \theta, \sigma, \rho, v_0$ puis déplacer le bloc de sauts le long de w_8 laisse les smiles cotés presque inchangés : l'indétermination devient visible.

5 Jeux de paramètres, enregistrement et export

Un format pour tous les jeux. Calibrations historiques, réglages enregistrés dans la page et sorties des projets de calibration partagent le format `calibration-a-la-main/jeux` : `jour`, `instantane`, `modele` (variance, sauts), `parametres`, `calibration` (origine, moteur, méthode, source), `donnees` (échéances, filtres), `pricer`, `perte`, `resultats`. Les 529 jeux calibrés embarqués (dossiers de calibration quotidienne, anciennes pages, chaîne quotidienne), dédoublonnés, sont rangés dans les jours.

Un jeu appartient à un modèle. Le menu « Point de départ » ne montre que les jeux du modèle actif : mes réglages, puis les calibrations du jour de la meilleure RMSE IV à la moins bonne, nommées « moteur · méthode · n éch. · perte · RMSE », puis les départs. Charger un jeu rétablit aussi ses échéances, son pricer et sa perte.

Enregistrement. Chaque réglage est écrit dans le navigateur et sur le disque : `resultats/<jour>/<variance>-<sauts>/NN_<nom>/` (`reglage.json`, `params.json`, `results.csv`, `df_used.csv`, `smile_grid.csv`, `smiles.png`), et ajouté à `resultats/jeux_enregistres.json` ; `resultats/<jour>/final/` reçoit le réglage marqué final.

Export filtré. Tous les jeux d'un modèle, sur tous les jours, filtrés par origine, nombre d'échéances, perte, période et RMSE IV maximale, tous ou le meilleur par jour.

Aide des modèles. Le bouton « Dynamique » affiche la dynamique risque-neutre du modèle actif (équations du future et de la variance, loi des sauts, compensateur, fonction caractéristique, contraintes, paramètres).

Jour du jour. Le bouton « Aujourd'hui » télécharge l'instantané chez Deribit ; le jour est gardé dans le navigateur (IndexedDB) et écrit en fichier `jours/<date>.js` dans le dossier de l'application, puis remis dans la liste à chaque lancement ; `build.py` l'intègre ensuite à la page.

Intensité nulle. Une intensité de sauts à 0 met la taille de ses sauts (μ_J, σ_J) à 0 et la fige ; elle reprend sa valeur précédente quand l'intensité redevient positive.

6 Données et reconstruction

```
cd calibration_a_la_main/sources
python ajouter_jours.py <instantanes>                          # jours absents
python ajouter_jours.py <instantanes> 2026-10-13               # jours choisis
python ajouter_calibrations.py historique.csv --ajouter   # jeux calibres de la chaine
    quotidienne
python telecharger_futures.py                                  # futures.js (poste connecte)
python build.py                                                # reassemble index.html et
    jeux.js
```

`ajouter_jours.py` prépare chaque instantané par `donnees_deribit.preparer_base` (chaîne complète nettoyée), marque le carnet de production (`preparer_carnet`) et y range les jeux calibrés du jour (`sources/calibrations.json`, format commun) et un point de départ générique. Sur un jour témoin, les quinze colonnes de données régénérées sont identiques à celles du fichier livré.

7 Validation

Le moteur JavaScript a été contrôlé contre les paquets Python : martingalité des 28 compositions, Black-Scholes, COS contre PROJ, PROJ bi-log-normal égal à `pricers_fourier.prix_proj` à $6 \cdot 10^{-6}$ \$, lois des quantiles (monotonie, rang).

8 Domaine de validité

- Optimiseur local : le résultat dépend du point de départ (pas de recherche globale dans `calibration_a_la_main`).
- Empilement de jours : partager v_0 n'est justifié que pour des jours rapprochés.
- Écriture de fichiers seulement dans Chrome et Edge (API d'accès au système de fichiers).